

Econometrics Softwares

Session 6: Introduction to Machine Learning

Mathilde Col & Jérémy Do Nascimento Miguel
University of Bordeaux

Regularized Linear regression models

Linear models

Recall the standard linear model is defined as follow:

$$y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \epsilon$$

Why we might want to use alternatives to OLS? Let's consider we want to run this model with n observations and p covariates

1. Prediction accuracy:

- If $n \gg p \Rightarrow$ OLS estimates have low variance
- If n is close to p , then OLS has high variability yielding to poor predictions and overfitting issue. Why? Many of the potential covariates will be correlated with each other $\Rightarrow$ not statistically significant. Should we keep them in or leave them out?
- If $p > n$, there is not a single linear solution but several resulting in poor Test MSE

2. Model interpretability:

- When $\times$ covariates are irrelevant, keeping them in the model increases the complexity while removing them increases the interpretability

Solutions? (i) *Subset selection* using theory to pick p ; (ii) *Shrinkage*: use all p but shrunk the estimated coefficient; (iii) *dimension-reduction* : e.g., pca

Shrinkage method: Ridge Regression

Shrinkage methods consists in fitting a model with all p covariates using a penalized estimate. Similar to least square, except that the coefficient are estimated by minimizing:

$$\hat{\beta}^R = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 + \lambda \sum_{j=1}^p \beta_j^2 = \text{RSS} + \lambda \sum_{j=1}^p \beta_j^2$$

with $\lambda > 0$: tuning parameter. Two different criteria

- Coefficient estimates fit the data well: min. the RSS
- The second term is a shrinkage penalty is small when $\beta_1, \dots, \beta_p$ are close to zero

λ controls the relative impact of these two on the reg. coeff. estimates

- $\lambda=0$, ridge regression = least square estimates
- $\lambda \rightarrow \infty$: shrinkage penalty $\uparrow$ and $\hat{\beta}^R \rightarrow 0$

Key is to select the good value of λ . Will see in Lab.

LASSO

Ridge drawback: includes all p predictors in the final model. Not a problem in prediction accuracy but it is for model interpretability. The most common used in economics is the **LASSO**.

LASSO = Least Absolute Shrinkage and Selection Operator

$$\beta_L = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 + \lambda \sum_{j=1}^p |\beta_j|$$

- $\hat{\beta}_{\lambda}^L$: LASSO coefficient which minimizes $RSS + \lambda \sum_{j=1}^p |\beta_j|$
- A linear regression with a penalty term, and a tuning parameter λ
- The penalty term shrinks some coefficients towards zero when λ is large enough
- It performs variable selection by shrinking some coeff. estimates to be exactly zero
- Ease model interpretation resulting in a **sparse model** (model with a subset of covariates)

Lasso vs Ridge?

Neither Lasso nor the ridge universally dominates the other, **but** some rules/patterns are well known.

- Lasso performs better in setting where a relatively small number of predictors have substantial coefficients and the remaining have very small or zero
- Ridge regression will perform better when the response is a function of many predictors, all with roughly same size

Issue: number of predictors that is related to the response is never known *a priori* for real data sets $\Rightarrow$ CV can be used to determine which one is the best

Selecting the tuning parameter

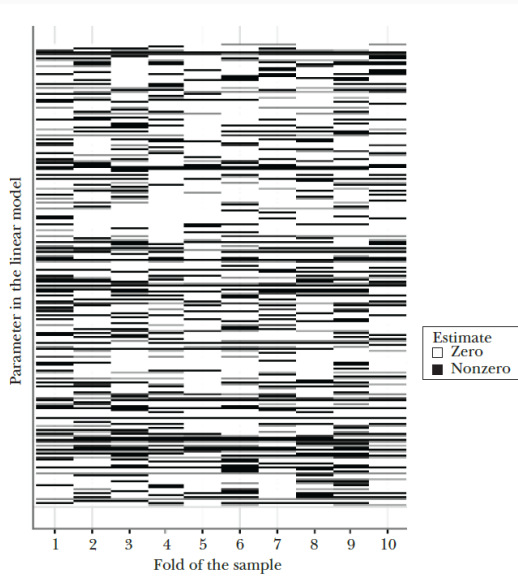
How to choose λ ? **Cross-validation** provides an easy approach:

- Choose a grid of λ values
- Compute the cross-validation error for each value of λ
- Select λ for which the CVE is smallest
- Re-fit the model using all of the available observations and the selected

LASSO: few elements to have in mind

- **LASSO is not scale invariant!** Changing the scale of the covariates changes the estimator. We should standardize any variable x to be mean-zero unit variance
- Machine learning has different goals than microeconometrics. Even when a ML looks like a regression, the interpretation of its coefficient estimates are different than OLS ones.
 - The coefficients are in general biased
 - Whether a covariate is included or excluded in the regression post-LASSO is meaningless

Interpreting the variables from LASSO? (Mullainathan et al., 2017)



Implementation in R

In R: we're going to need the “glmnet” for regularized generalized linear models

```
fit <- rlassoEffects(x = X, Y = Y , d = D , method='double selection')
```

Inference with selection among many controls

Consider a linear model where a treatment variable, d_i , is exogenous

$$y_i = \alpha d_i + x_i' \theta + r_{yi} + \eta_i \quad (1)$$

α : Treatment effect on the outcome .

Naive approaches

1. Run Lasso while forcing d_i to remain in the OLS:
 - Problem: Lasso targets prediction and will drop any variables highly correlated to the treatment. **OVB risk**
 - Omit relationship between treatment variable and controls: can model this reduced form
2. Use only eq. from previous slide or the reduced form
 - Assume: no errors in variable selection
 - Issue: tend to pick variables with larger value correlated with y_i and miss variables which have large correlation with d_i

Post-double selection LASSO

belloni2014inference's paper. They consider a regression model with a treatment indicator and control variables on the RHS:

- We are in a case with large number of *potential* controls $(x_1, \dots, x_p)$. Potentially in a case where $p \gg n$
- Objective is to select a small sub-sample of variables that matter, s

They propose a “double selection procedure” involving 3 steps:

1. Select covariates for outcome y_i using LASSO (first LASSO)
2. Select covariates for treatment status D_i using LASSO (second LASSO)
3. Use covariates relevant in one of the cases as control variables

PDS-LASSO: formal framework

$$y_i = D_i\alpha_0 + g(z_i) + \epsilon_i \quad E[\epsilon_i|z_i, D_i] = 0 \quad (2a)$$

$$D_i = m(z_i) + \eta_i \quad E[\eta_i|z_i] = 0 \quad (2b)$$

y_i is the outcome, D_i the treatment variable, z_i represents confounding factors, which affect D_i via $m(z_i)$ and y_i via $g(z_i)$.

However, the form of $g()$ and the identity of z are unknown. Therefore, we approximate (1a) and (1b) using a linear combination of all potential control variables (x_i):

$$y_i = D_i\alpha_0 + x_i'\beta_{g0} + r_{gi} + \epsilon_i \quad (3a)$$

$$D_i = x_i'\beta_{m0} + r_{mi} + \eta_i \quad (3b)$$

$x_i'\beta_{g0}$ and $x_i'\beta_{m0}$ are approximations to $g(z_i)$ and $m(z_i)$; r are the corresponding approximation errors.

PDS Lasso: The recipe

Practically, PDS is implemented in three steps:

1. Lasso Y_i on X_i , collect selected controls in X_i^Y
2. Lasso D_i on X_i , collect selected controls in X_i^D
3. Regress Y_i on D_i and $X_i^Y \cup X_i^D$

Caveats and considerations:

- Standardizing controls pre-lasso is important
- Cross-validation works fine for selection λ
- Inference: Just use robust standard errors from step 3

Issues in high-dimension settings

What can go wrong when one applies techniques not intended for high-dimensional settings?

- When $p \gg n$ or even p slightly larger than $n \rightarrow$ Cannot perform OLS
 - Reason: OLS yields to a set of coefficient estimates that result in a perfect fit to the data, such that the residuals are zero
 - Possible to perfectly fit the training data in the high-dimensional setting, the resulting linear model will perform extremely poorly on an independent test set
 - Too flexible and overfit the data
- Adding more variables $\uparrow R^2$ and $\downarrow$ Train MSE even though the new variables are unrelated to y (same way $\uparrow$ Test MSE because $\uparrow$ variance of coeff.)

Implementation in R

We already saw how to implement a lasso, now let's see for the PDS Lasso. You should have the packages “glmnet” and “hdm”.

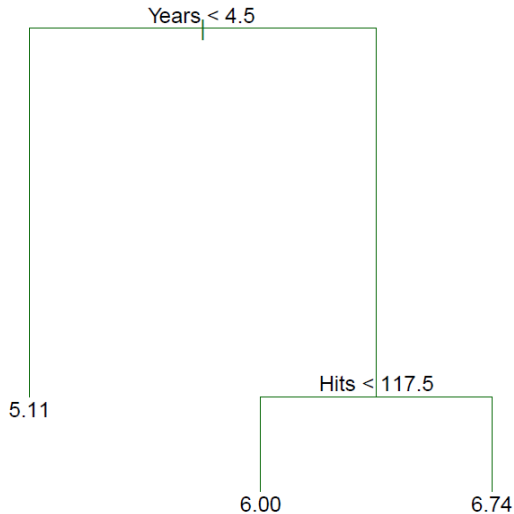
```
fit <- rlassoEffects(x = X, Y = Y , d = D , method='double selection')
```

Tree-based methods

Intuition:

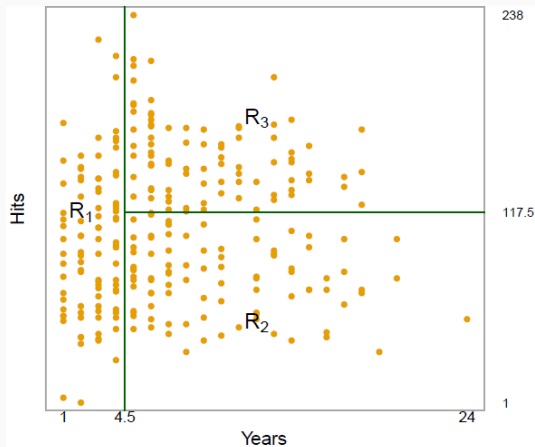
- Segment the set of possible values for $X_1, \dots, X_p$ (i.e., predictor space) into J distinct and not overlapping regions $R_1, \dots, R_J$
- Assign the region average outcome predicted value (region mean) for every observation in a region

A regression tree and some terminology



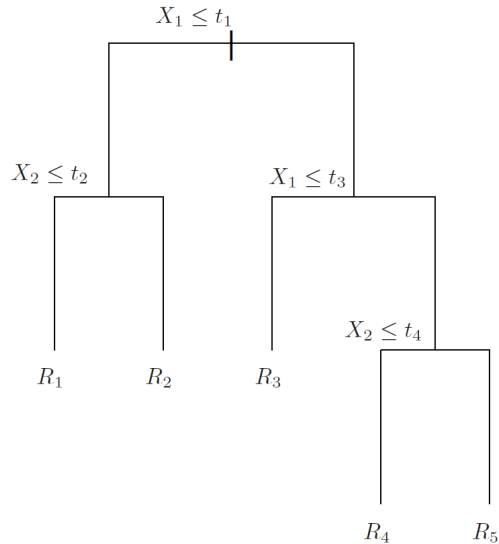
From Hastie et al. (2009)

Representing a tree formally



From Hastie et al. (2009)

Basic approach to grow a tree



Growing a regression tree

1. Find boxes $R_1, \dots, R_J$ minimizing $RSS = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$
 - Problem: infeasible to consider all possible partitions. Solution is to implement a recursive binary splitting to grow the largest tree with the training data
 - Top-down: starts at the top of the tree, then successively splits into new branches
 - Greedy: best split is made at each step.
2. Repeat the procedure at each node until a stopping criterion is reached:
 - Minimum # of observations in each terminal node R_m or maximum tree depth
3. Once all regions have been created, $\hat{f}(x)$ is equal to the average outcome within each region R_m that the x of an observation fall into

Practice: start with all the data, select the covariates X_j and cutpoint s to split the regions leading to highest possible reduction in RSS.

Regression trees: when to stop??

Similarly to previous ML algorithms, we are facing the bias-variance trade-off

- Larger trees (= $\times$ regions): minimize bias but have large variance $\Rightarrow$ overfitting and interpretation complexity
- Smaller trees: higher bias but have lower variance $\Rightarrow$ easier to interpret.

Strategy is to grow a very large tree T_0 and **pruning** it in subtrees:

- Sequence of trees indexed by a non negative tuning parameter α
- For each α there corresponds a subtree $T \subset T_0$ such that the following is minimized:

$$\sum_{m=1}^{|T|} \sum_{\mathbf{x}_1 \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

with R_m is the rectangle links to the m^{th} terminal node, α controls the tradeoff between subtree complexity and its fit to the training data, and $|T|$ is the number of terminal node in tree T

Regression trees: when to stop??

$$\sum_{m=1}^{|T|} \sum_{\mathbf{x}_1 \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T| \quad (4)$$

with R_m is the rectangle links to the m^{th} terminal node, α controls the tradeoff between subtree complexity and its fit to the training data, and $|T|$ is the number of terminal node in tree T

- $\alpha = 0$: subtree = tree, why? 4 = training error
- α min. 4 with small subtrees and branches get pruned in a nested and predictable fashion
- $\approx$ to Lasso

Advantages of growing trees: interpretable and handle qualitative variables

Linear model or trees?

- Linear model:

$$f(X) = \beta_0 + \sum X_j \beta_j$$

- Trees:

$$f(X) = \beta_0 + \sum c_m \cdot 1_{(X \in R_m)}$$

Assess relative performance
estimating test error by CV

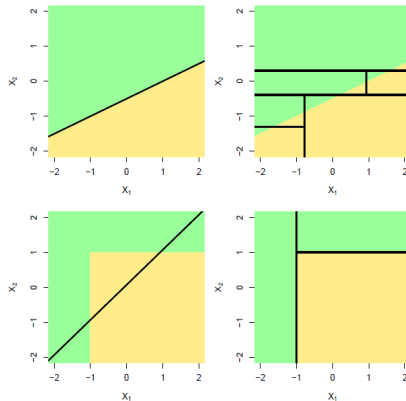


FIGURE 8.7. Top Row: A two-dimensional classification example in which the true decision boundary is linear, and is indicated by the shaded regions. A classical approach that assumes a linear boundary (left) will outperform a decision tree that performs splits parallel to the axes (right). Bottom Row: Here the true decision boundary is non-linear. Here a linear model is unable to capture the true decision boundary (left), whereas a decision tree is successful (right).

From growing trees to planting forests

Growing a single tree has some drawbacks:

- Not as accurate as other methods
- Not as robust (high variance): if we split the training data at random and fit tree to both halves, the results that we get could be quite different

Bagging intuition: the average of random variables has lower variance than any single of them.

- It takes multiple bootstrap samples from the training set, build a separate prediction model on each of them and average them: $\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$
- Assess performance using the **out-of-bag observations** ($\approx 1/3$ of the sample) to predict y for the i^{th} observation using each of the trees in which it was OOB, yields to $B/3$ predictions for the i^{th} obs.
- Average each of them to compute the the out-of-bag error (i.e., test MSE).

Drawbacks:

- More complex and less interpretable than trees
solution: record the total amount that the RSS is decreased due to splits over a given predictor, averaged over all B trees.
- $\hat{f}^b$ might be highly correlated in practice $\Rightarrow$ use random forests to decorrelates the trees

Variables importance plot

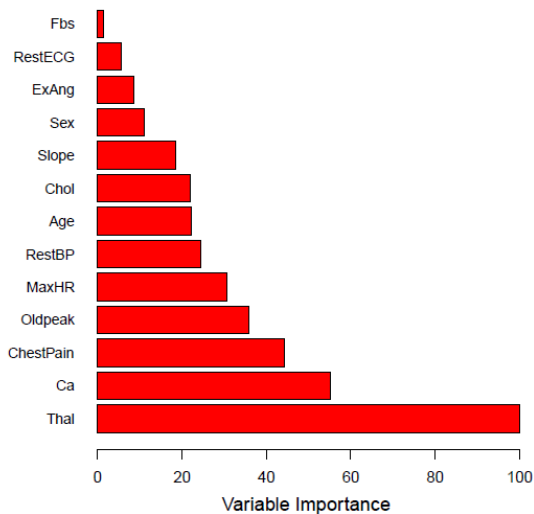


FIGURE 8.9 A variable importance plot for the Heart dataset. Variables

Random forest

Random Forest: randomly restrict which splits of the tree are allowed at each node. At each split, only a random subsample of covariates m is considered. Then, one covariate is chosen among m . A fresh sample is drawn at each split.

- RF is not even allowed to consider a majority of the available predictors
- Why? Suppose 1 very strong p and a large number of not strong p . Then in the collection of bagged- trees, most of them will use the strong at the top $\Rightarrow$ all of the bagged trees will look quite similar

$\Rightarrow \hat{f}^b$ will be less correlated which reduces variance

- Usually, we pick $m \approx \sqrt{p}$ to avoid strong covariates to be picked at the top of each tree.
- On average $\frac{p-m}{m}$ of splits not consider a covariate
- Main difference with bagging: choice of subset m
- Note that RF=bagging if $p = m$
- As with bagging, it will not overfit if we increase $B \Rightarrow$ use large B

While **bagging** involves creating *multiple copies* of the original training data, **boosting** *grows each tree sequentially* using information from previously grown trees. Each tree is fit on a modified version of the original data set.

1. Fit a decision tree to the residual from the model. It fits a tree using the current residuals and not the outcome as dependent variable
2. Add this new decision tree into the fitted function and update the residuals
3. Grows a new tree using the updated residual from the previous fitted function.

Small trees are grown successively such that it improves slowly the fitted function in poor performance area.

Boosting for regression trees

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in the training set.
2. For $b = 1, 2, \dots, B$, repeat:
 - (a) Fit a tree $\hat{f}^b$ with d splits ($d+1$ terminal nodes) to the training data (X, r) .
 - (b) Update $\hat{f}$ by adding in a shrunk version of the new tree:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \quad (8.10)$$

- (c) Update the residuals,

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i). \quad (8.11)$$

3. Output the boosted model,

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x). \quad (8.12)$$

Parameters to consider

Three tuning parameters:

- Number of trees B : overfit if too large. Use cross-validation to pick B
- Shrinkage parameter λ : small positive number which controls the boosting learning rate. The smaller it is, the larger B should be
- Number of splits d in each tree: controls the complexity. Works well with $d = 1$

Unpack the treatment effect
heterogeneity

Basic causal inference summary

Target is to estimate the average treatment

$$ATE = E[Y_i(1) - Y_i(0)] = E[\tau_i]$$

The key identifying assumption:

$$(Y_i(0), Y_i(1)) \perp D_i | X_i$$

Heterogeneous Treatment Effect

Uncovering treatment effect heterogeneity with respect to some covariates

- Individuals benefit most from a treatment?
- How do treatment effect change with covariates?

Estimate the overall average effect:

$$Y_i = \tau D_i + \epsilon_i$$

Explore heterogeneity by gender:

$$Y_i = \tau^{male} D_i + \beta \text{Female}_i + \gamma D_i \times \text{Female}_i + \epsilon_i$$

More generally,

$$Y_i = \tau D_i + \beta X_i' + \gamma D_i \times X_i' + \epsilon_i$$
$$\tau(x) = \tau + x' \gamma$$

Conditional average treatment effect (CATE): $\tau(x) = E[\tau_i | X_i = x]$

Challenge with traditional heterogeneity analysis

$$Y_i = \tau D_i + \beta X_i' + \gamma D_i \times X_i' + \epsilon_i$$

- Functional form: treatment effects may not vary linearly with X_i
- Curse of dimensionality
- Luckily we can rely on ML to solve these issues

Heterogeneous treatment effects in randomized experiments

In most of our analysis, we want to know how treatment effects vary with covariates to explore mechanisms or predict the effect in a specific subpopulation

Problem? We often have many covariates and can find significant effects by "chance" = cherry picking

- Solution 1: register a pre-analysis plan (PAP). But that is partially satisfactory, we are throwing away lots of data we are collecting
- Solution 2: use machine learning to guide prediction as complement to theory

My advice would be to combine both: identify sub-categories of interest that make sense from a theoretical standpoint in a PAP, and dig deeper with ML

Problem in causal tree approaches: inference does not account for uncertainty in covariates binning

- Why? Rely on split to choose the bin, then CATE within those bins work roughly as well as regression approach
- But fails to consider bins variation across samples

Generic ML inference: the set up

$Y(1)$ and $Y(0)$ are the potential outcomes in the treatment and control group. Z is a vector of covariates. We assume that:

$$Y(1), Y(0) \perp D|Z$$

The observed outcome is $Y = DY(1) + (1 - D)Y(0)$ which given D, Z can be rewritten such as:

$$Y = b_0(Z) + Ds_0(Z) + U, \quad E[U|Z, D] = 0$$

The main causal functions are the baseline conditional average:

$$b_0(Z) = E[Y(0)|Z]$$

and the conditional average treatment effect:

$$s_0(Z) = E[Y_1 - Y_0|Z]$$

We are interested in $s_0(Z)$, the CATE, but **we cannot observe it**

The problem and the solution

We do not know how to do uniformly valid confidence based on z . Therefore, Chernozhukov et al. (2020) propose to estimate and make inference on *key features* of $s_0(Z)$ rather than $s_0(Z)$.

- Identify how much heterogeneity there is in the estimates and then find out those groups characteristics with HE
- Randomly split sample into main and auxiliary sample
- Apply ML in $Data_A$ to estimate control mean, $B(Z)$, and treatment effect, $S(Z)$
- Use test sample to predict actual values

Key features of $s_0(Z)$

1. Best Linear Predictor of $s_0(Z)$ given $S(Z)$
2. Sorted Group Average Treatment Effects (GATES)
3. Classification Analysis (CLAN)

Best Linear Predictor (BLP)

The BLP of $s_0(Z)$ given $S(Z)$ is given by:

$$BLP(s_0(Z)|S(Z)) = \beta_1 + \beta_2(S(Z) - ES(Z))$$

but we can't run the regression of $s_0(Z)$ on $S(Z)$, why?

Best Linear Predictor

Instead, we can run the following model in $Data_A$

$$Y = \alpha' X_1 + \beta_1(D - p(Z)) + \beta_2(D - p(Z)(S(Z) - ES) + \epsilon$$

- X_1 is a vector of ones, $B(Z)$, and variables used to improve precision (e.g., district FE)
- $p(Z)$ is the probability of treatment conditional on covariates

We obtain β_{*1} (ATE) and β_{*2} , the treatment effect heterogeneity on the proxy predictor (HET)

Then, we can estimate the same model using $Data_M$ and test for:

$$H0 : \beta_2 = 0$$

$H0$ corresponds to the combined hypothesis that:

1. No treatment effect heterogeneity based on Z ($s_0(Z) = s$)
2. or $S(Z)$ is completely noise, uncorrelated to $s_0(Z)$

Moving on from BLP

If we reject $H_0 : \beta_2 = 0$ from the BLP, we have identified heterogeneity. Now we would like to know which covariates drive it. Main issue: high-dimensional settings

Sorted Group Average Treatment Effects (GATES)

Can use the covariates in Z to **GROUP** individuals together into k groups according to the ATE size $S(Z)$:

$$\gamma_k = E[s_0(Z)|G_k], \quad k = 1, \dots, K$$

Practically, we estimate the following specification:

$$Y = \alpha'X_1 + \sum_{k=1}^K \gamma_k (D - p(Z)) G_k + \epsilon$$

with G_k is a dummy variable for being in group k and test for

$$H_0 : \gamma_k - \gamma_1 = 0$$

Test whether the GATES differ between the most and least affected groups

Issue we still have not solve: we observe heterogeneity in treatment effects across groups but we still do not know which covariates drive these results

Classification Analysis (CLAN)

Focus on the "least affected group" G_1 and the "most affected group" G_K

Objective: test for differences along covariates comprising Z between G_1 and G_K

- Let $g(Y, Z)$ be a vector of characteristic of a unit
- Parameters of interest are the avg. characteristics (e.g., age, income) of the G_1 and G_K :

$$\delta_1 = E[g(Y, Z)|G_1] \quad \text{and} \quad \delta_K = E[g(Y, Z)|G_K]$$

Test the following hypothesis:

$$H_0 : \delta_K - \delta_1 = 0.$$

Rejecting H_0 for a given characteristic in Z means there is a difference between the most and least affected groups for this characteristic

Inference

Let θ denote the generic parameter of interest, for instance:

- $\theta = \beta_2$ the heterogeneity loading parameter
- $\theta = \beta_1 + \beta_2(S(z) - ES)$ BLP of $s_0(Z)$
- $\theta = \gamma_K$ is the expectation of $s_0(Z)$ for a given group
- $\theta = \gamma_K - \gamma_1$ difference in expectation of $s_0(Z)$ between the most and least affect gps
- $\theta = \delta_K - \delta_1$ difference in expectation of the characteristics

Two sources of uncertainty:

1. Estimation uncertainty regarding θ conditional on the data split (as we seen before)
2. Uncertainty induced by the data splitting (new)

⇒ develop confidence intervals taking into account both sources: repeated random splits of the initial data into $Data_M$ and $Data_A$. Compute individual split CI, θ , and p-value. Then compute sample median over a certain number of random splits (rather than all possible splits).

Empirical application

Application using Crepon et al., (2015)

Experiment in Morocco estimating the effect of access to microfinance services on financial and non-financial outcomes

- 162 *villages* in rural Morocco, divided into 81 *pairs*, 5,551 households
- Within each pair: 1 treatment and 1 control village
- Treatment: opening of a microfinance institution
- Started in 2006 with follow-up surveys in 2009
- Use stratified sample splitting with village pairs as strata
- Variables:
 - Y = financial and non-financial outcomes. We are focusing on
 - D = indicator of being in a village where a microfinance institution opened
 - Z = vector of household characteristics, and pair fixed effects

Best Linear Predictors of Conditional Average Treatment Effect

Experiment in Morocco estimating the effect of access to microfinance services on financial and non-financial outcomes

- 162 *villages* in rural Morocco, divided into 81 *pairs*, 5,551 households
- Within each pair: 1 treatment and 1 control village
- Treatment: opening of a microfinance institution
- Started in 2006 with follow-up surveys in 2009
- Use stratified sample splitting with village pairs as strata
- Variables:
 - Y = financial and non-financial outcomes. We are focusing on total amount borrowed
 - D = indicator of being in a village where a microfinance institution opened
 - Z = vector of household characteristics, and pair fixed effects
- Rely on R package "Generic_ML"

Main results: low take-up (17% in T group) and significant effect on total borrowing: ATE of MAD 1,193 (Table 2, col 7)

Best Linear Predictor (BLP)

Recall: BLP estimates some β_1 and β_2 via OLS, with $\beta_1 = E_{s0}(Z)$ is the ATE and $\beta_2 \neq 0$ if there is heterogeneity in $s0(Z)$ and $S(Z)$ predicts it well

BLP generic targets

	Estimate	CI lower	CI upper	p value
beta.1	1029.7045	147.4776	1756.1096	0.002
beta.2	0.3270	0.0344	0.6431	0.018

Confidence level of confidence interval [CI lower, CI upper]: 90 %

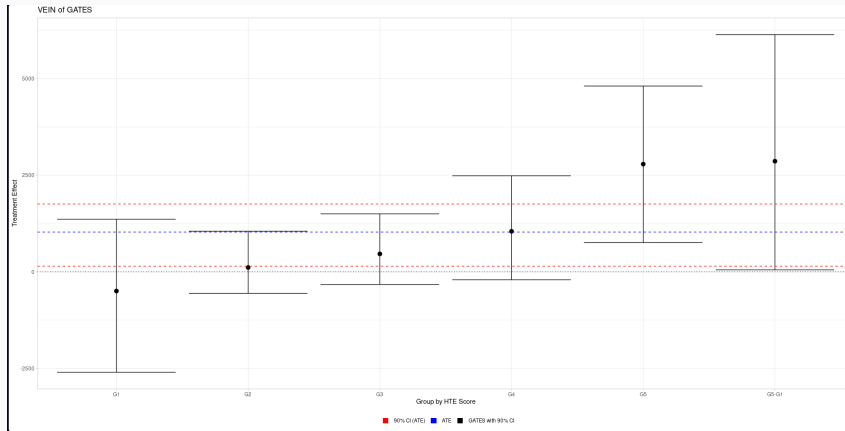
Sorted Group Average Treatment Effects (GATES)

Build groups, here 5 and estimate the group-ATE $\gamma_k = E[s_0(Z)|G_k]$ via OLS

```
---  
              Estimate CI lower CI upper p value  
gamma.1      -496.68 -2600.26  1361.75  0.626  
gamma.2       112.78  -556.54  1051.31  0.670  
gamma.3       464.62  -328.14  1501.24  0.023  
gamma.4      1049.35  -205.04  2485.63  0.049  
gamma.5      2787.61   757.60  4809.41  0.007  
gamma.5-gamma.1 2863.38    51.56  6140.28  0.021  
---  
Confidence level of confidence interval [CI lower, CI upper]: 90 %
```

Sorted Group Average Treatment Effects (GATES)

Build groups, here 5 and estimate the group-ATE $\gamma_k = E[s_0(Z)|G_k]$ via OLS



Classification Analysis (CLAN)

Observed within-group averages, δ_k , of a given variable for groups G_k . Here household head age.

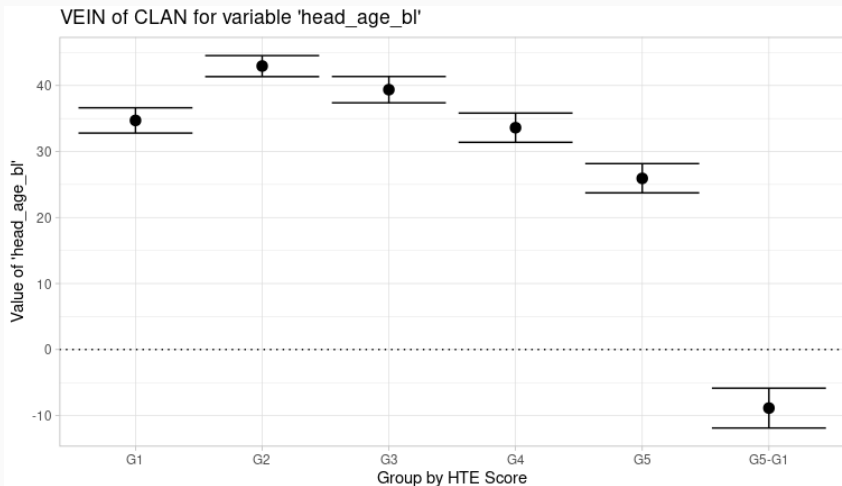
```
CLAN generic targets for variable 'head_age_b1'
```

```
---
```

	Estimate	CI lower	CI upper	p value
delta.1	34.705	32.789	36.621	0
delta.2	42.946	41.342	44.521	0
delta.3	39.365	37.373	41.357	0
delta.4	33.603	31.390	35.816	0
delta.5	25.924	23.734	28.164	0
delta.5-delta.1	-8.871	-11.887	-5.854	0

Classification Analysis (CLAN)

Observed within-group averages, δ_k , of a given variable for groups G_k . Here household head age.



Exercise 1

Using the AGSS_15 data from the previous section:

- Run again the same OLS regression as in session 2
- Run a PDS Lasso using all the covariates available. What is the MSE? How does your coefficient of interest look now?
- Using a random forest, which variables are the most predictive of the value of our outcome of interest?

Exercise 2

Use data (Triyana_2016_price_women_clean.csv) from a randomized experiment in Indonesia called Program Keluarga Harapan (PKH). PKH was a conditional cash transfer program designed to improve child health. See [Tyriana \(2016\)](#)

- The treatment variable is “pkh_kec_ever”
- Estimate the average treatment effects of the treatment on children’s birth weight (**bw**), clustering the standard errors using “loc_id”
- Apply Generic Machine learning to measure whether the treatment has heterogeneous effects on the outcome
 - Use lasso and randomforest algorithms
 - Use all the covariates available in the dataset, note: do not forget to standardize continuous variables and transform categorical variables into dummies
 - Use 5 groups for measuring CATE
- Estimates BLP, GATES, and CLAN. Do CLAN for “educ” and “goat”